

Parser di Numeri in Virgola Mobile con Macchina a Stati di Mealy

Indice

1. [Introduzione](#)
2. [Concetti Chiave](#)
3. [Architettura della Macchina a Stati](#)
4. [Descrizione degli Stati](#)
5. [Tabella di Transizione](#)
6. [Implementazione](#)
7. [Funzioni Utility](#)
8. [Test e Validazione](#)
9. [Differenze Mealy vs Moore](#)
10. [Ottimizzazioni](#)

Introduzione

Questo documento analizza un **parser di numeri in virgola mobile** che utilizza una **macchina a stati di Mealy** per convertire stringhe di testo in valori numerici double.

Casi di uso:

- Parsing di input utente
- Validazione di numeri in formati diversi
- Analisi di file CSV o di configurazione

Formato supportato:

```
[±][cifre][.[cifre]][e|E[±]cifre]
```

Esempi validi:

- 12, -12, +12
- 12.34, 0.12
- 12e3 (12000), 12.34E-2 (0.1234)
- 12.34 (con spazi)

Esempi invalidi:

- -+12 (doppio segno)

- .14 (punto all'inizio)
- 12. (punto alla fine)
- 12e (esponente vuoto)

Concetti Chiave

Macchina a Stati (Finite State Machine - FSM)

Una macchina a stati è un modello computazionale che:

1. Risiede sempre in uno **stato** specifico
2. Riceve un **input** alla volta
3. Transisce a un **nuovo stato** basato su (stato attuale, input)
4. Può produrre un **output** durante la transizione

Mealy vs Moore

Aspetto	Mealy	Moore
Output dipende da	(stato, input)	solo dallo stato
Output quando	sulla transizione	nello stato
Numero stati	spesso meno	spesso più
Latenza	0 cicli	1 ciclo

In questo parser usiamo **Mealy** perché l'azione (es. `sign = -1`) dipende sia dallo stato che dal carattere ricevuto.

Accettore vs Traduttore

- **Accettore**: risponde "sì" o "no" (es. "è un numero valido?")
- **Traduttore**: trasforma input in output (es. "converti string a double")

Questo parser è un **traduttore**: accetta una stringa e restituisce un numero.

Architettura della Macchina a Stati

Diagramma di Flusso

INIZIO PARSING

INPUT: "12.34e-5" + marker speciale "~"

,

$\frac{1}{4}$

[A] START

Aspetta: $\pm$ 0

,

4

se '+' o '-'

se

$\frac{1}{4}$

$\frac{1}{4}$

[B] SIGN

Segno letto

[C] MANTISSA INT

Cifre intere

,

,

4

se '.'

se o 'e'

$\frac{1}{4}$

$\frac{1}{4}$

[D] PUNTO

Punto visto

[E] MANTISSA

Cifre decimali

,

,

4

se 'e'

se

$\frac{1}{4}$

$\frac{1}{4}$

[F] ESPON

Espon. visto

[G] ESPON_SIGN

Segno espon.

,

,

4

se

se '~'

1/4

[H] EXPON
Cifre espon.

(se '~')

1/4

[I] ACCEPT
Numero OK!

1/4

[I] ACCEPT
Numero OK!

Se in qualunque momento arriviamo a una transizione non valida:

1/4

[Z] ERROR
Numero KO

Descrizione degli Stati

Stato	Nome	Descrizione	Input Accettati	Azione
A	START	Inizio parsing	+ , - ``	Imposta segno se ±
B	SIGN	Segno già letto	``	Continua a B o va a C
C	MANTISSA INT	Cifre intere	`,` . , e , E	Accumula cifre intere
D	PUNTO	Punto decimale visto	``	Prepara accu- mulo decimali

E	MANTISSA FRAZ	Cifre decimali	`,` e , E	Accumula cifre decimali
F	ESPON VISTO	Notazione esponenziale	+ , - `,`	Imposta segno esponente
G	ESPON SIGN	Segno esponente	`,`	Continua a G o H
H	ESPON	Cifre esponente	`,`	Accumula esponente
I	ACCEPT	Stato finale accettante	-	Restituisci il numero
Z	ERROR	Stato di errore	-	Restituisci NaN

Tabella di Transizione

La tabella di transizione definisce tutte le transizioni possibili. Formato: (statoattuale, input) (stato_nuovo, azione)

Transizioni Valide

Stato A (START):

```
(A, '+') → (B, none)
(A, '-') → (B, sign = -1)
(A, ') → (C, value += digit)
```

Stato B (SIGN):

```
(B, ') → (C, value += digit)
```

Stato C (MANTISSA INT):

```
(C, )' (C, value += digit)
(C, '.') ' (D, none)
(C, 'e'/'E') (F, none)
```

Stato D (PUNTO):

```
(D, )' (E, value += fvalue*digit, fvalue /= 10)
```

Stato E (MANTISSA FRAZ):

```
(E, )' (E, value += fvalue*digit, fvalue /= 10)
(E, 'e'/'E') (F, none)
```

Stato F (ESPON VISTO):

```
(F, '+') ' (G, none)
(F, '-') ' (G, eSign = -1)
(F, )' (H, exponent += digit)
```

Stato G (ESPON SIGN):

```
(G, )' (H, exponent += digit)
```

Stato H (ESPON):

```
(H, )' (H, exponent += digit)
```

Stato Finale:

```
(C, '~') ' (I, none) # Numero intero senza decimali
(E, '~') ' (I, none) # Numero con decimali
(H, '~') ' (I, none) # Numero con esponente
```

Transizioni di Errore:

Qualunque altro input in qualunque stato ' (Z, none)

Matrice Visualizzata

+	-	.	e/E	~	altri		
A	B [†]	B [†]	Cz	Z	Z	Z	Z
B	Z	Z	Cz	Z	Z	Z	Z
C	Z	Z	Cz	D	F	I	Z
D	Z	Z	Ez	Z	Z	Z	Z
E	Z	Z	Ez	Z	F	I	Z
F	G [†]	G [†]	Hz	Z	Z	Z	Z
G	Z	Z	Hz	Z	Z	Z	Z
H	Z	Z	Hz	Z	Z	I	Z
I	-	-	-	-	-	-	-
Z	-	-	-	-	-	-	-

Legenda:

[†] = imposta segno negativo

z = accumula cifra

I = stato accettante

Z = stato errore

Implementazione

Variabili di Stato

```
State currentState = A;           // Stato attuale della macchina
double sign = 1;                  // Segno del numero (+1 o -1)
double value = 0;                 // Valore accumulato (intera + frazionaria)
double exponent = 0;              // Valore esponente
double eSign = 1;                 // Segno esponente (+1 o -1)
double fvalue = 0.1;              // Moltiplicatore frazionario (0.1, 0.01,
...)
```

Logica Principale

```
double ParseDouble(string input) {
// Inizializzazione variabili
State currentState = A;
double sign = 1, value = 0, exponent = 0, eSign = 1, fvalue = 0.1;
```

```

// Per ogni carattere (incluso marker '~')
foreach (var ch in input.Trim() + "~") {
    int digit = ch - '0'; // Converti char a int

    // Tabella di transizione
    (currentState, azione) = (currentState, ch) switch {
        (A, '+') => (B, none),
        (A, '-') => (B, () => sign = -1),
        // ... altre transizioni ...
        (C or E or H, '~') => (I, none),
        _ => (Z, none),
    };

    // Esegui azione
    azione();
}

// Restituisci il risultato
if (currentState == I) {
    return sign * value * Math.Pow(10, eSign * exponent);
}
return NaN;
}

```

Esempio di Esecuzione

Input: "-12.34e2"

Passo	Char	Stato Attuale	Transizione	Stato Nuovo	Azioni	Valori
1	'.'	A	(A, '-')	B	sign = -1	sign=-1
2	'1'	B	(B, '0-9')	C	value += 1	value=1
3	'2'	C	(C, '0-9')	C	value += 2	value=12
4	':'	C	(C, ':')	D	-	-
5	'3'	D	(D, '0-9')	E	value += 0.3, fvalue=0.01	value=12.3
6	'4'	E	(E, '0-9')	E	value += 0.04, fvalue=0.001	value=12.34

'e'	E	(E, 'e')	F	-	-
8	'2'	F	(F, '0-9')	H	exponent = 2
9	'~'	H	(H, '~')	I	-
10	-	I	-	-	Accetta!
					-12.34 × 10 ² = -1234

Funzioni Utility

Validazione

```

/// <summary>
/// Verifica se una stringa rappresenta un numero double valido.
/// </summary>
bool IsValidDouble(string input) {
    double result = ParseDouble(input);
    return !double.IsNaN(result);
}

```

Utilizzo:

```

Console.WriteLine(IsValidDouble("12.34"));    // true
Console.WriteLine(IsValidDouble(".14"));      // false
Console.WriteLine(IsValidDouble("12e"));      // false

```

Contare Cifre Decimali

```

/// <summary>
/// Conta il numero di cifre decimali in una stringa.
/// </summary>
int CountDecimalPlaces(string number) {
    int dotIndex = number.IndexOf('.');
    if (dotIndex == -1) return 0;

    // Estrai la parte dopo il punto
    string fractional = number.Substring(dotIndex + 1);
}

```

```
// Scarta caratteri non-numerici (es. 'e')
int eIndex = fractional.IndexOfAny(new[] { 'e', 'E' });
if (eIndex != -1) {
    fractional = fractional.Substring(0, eIndex);
}

return fractional.Length;
}
```

Utilizzo:

```
CountDecimalPlaces("12.345");    // 3
CountDecimalPlaces("12.34e2");    // 2 (cifre dopo punto, prima di 'e')
CountDecimalPlaces("12");         // 0
```

Formattazione Scientifica

```
/// <summary>
/// Converti un numero a notazione scientifica.
/// </summary>
string ToScientific(double value, int exponent) {
    return $"{value:G}e{exponent:+0;-0}";
}
```

Utilizzo:

```
ToScientific(12.34, -2);    // "12.34e-2"
ToScientific(1, 3);         // "1e+3"
```

Estrazione Componenti

```
/// <summary>
/// Estrai segno, mantissa ed esponente da una stringa.
/// </summary>
(bool isNegative, string mantissa, string exponent) ParseComponents(string input) {
    bool isNegative = input.StartsWith('-');

    // Estrai mantissa (tutto prima di 'e' o 'E')
    int eIndex = input.IndexOfAny(new[] { 'e', 'E' });
```

```

    string mantissa = (eIndex == -1)
    ? input.Replace("-", "").Replace("+", "")
    : input.Substring(0, eIndex).Replace("-", "").Replace("+", "");

    // Estrai esponente (tutto dopo 'e' o 'E')
    string exponent = (eIndex == -1)
    ? "0"
    : input.Substring(eIndex + 1);

    return (isNegative, mantissa, exponent);
}

```

Utilizzo:

```

var (neg, mant, exp) = ParseComponents("-12.34e-2");
// neg: true, mant: "12.34", exp: "-2"

```

Test e Validazione

Casi di Test

```

Dictionary<string, double> testCases = new () {
    // Interi
    ["12"] = 12,
    ["-12"] = -12,
    ["+12"] = 12,

    // Decimali
    ["12.34"] = 12.34,
    ["0.12"] = 0.12,

    // Scientifici
    ["12e3"] = 12000,
    ["12.34e-2"] = 0.1234,

    // Spazi
    [" 12.34 "] = 12.34,

    // Invalidi
    ["-+12"] = double.NaN,
    [".14"] = double.NaN,
    ["12."] = double.NaN,
    ["12e"] = double.NaN,
}

```

```
["e4"] = double.NaN,  
};
```

Esecuzione Test

```
int passed = 0, failed = 0;  
  
foreach (var (input, expected) in testCases) {  
    double result = ParseDouble(input);  
    bool success = Same(expected, result);  
  
    if (success) passed++;  
    else failed++;  
  
    Console.ForegroundColor = success ? ConsoleColor.Green : ConsoleColor.Red;  
    Console.WriteLine($"[{(success ? "" : "")}] \"{input}\" ' {result}");  
    Console.ResetColor();  
}  
  
Console.WriteLine($"\\nRisultati: {passed}/{testCases.Count} passati");
```

Funzione Helper di Confronto

```
bool Same(double a, double b) {  
    if (double.IsNaN(a)) return double.IsNaN(b);  
    return Math.Abs(a - b) < 1e-6; // Tolleranza per errori floating-point  
}
```

Differenze Mealy vs Moore

Macchina di Mealy (Questo Parser)

Caratteristiche:

- Output prodotto sulla **transizione**
- Output dipende da (stato attuale, input)
- Numero di stati: **minore**
- Implementazione: switch expression basata su (stato, input)

Diagramma:

```
input
[A] -----' [B]
azione
```

Vantaggi:

Meno stati
Output immediato
Più efficiente in termini di memoria

Svantaggi:

Output meno prevedibile (dipende sia da stato che input)
Più difficile da leggere/comprendere

Macchina di Moore (Alternativa)

Caratteristiche:

- Output prodotto nello **stato**
- Output dipende solo dallo stato attuale
- Numero di stati: **maggiore**
- Implementazione: for each char, ottieni output dallo stato

Diagramma:

```
input
[A] -----' [B]
"           "
[out_A]    [out_B]
```

Vantaggi:

Output prevedibile
Più facile da comprendere

Svantaggi:

Più stati richiesti
Latenza di un ciclo

Confronto: Parsing di "-12"

Mealy (questo parser):

```
Passo 1: (A, '-') ' (B, sign=-1)    # Output sulla transizione
Passo 2: (B, '1') ' (C, value+=1)   # Output sulla transizione
Passo 3: (C, '2') ' (C, value+=2)   # Output sulla transizione
Passo 4: (C, '~') ' (I, return)
```

Moore (alternativa):

```
Passo 1: A ' (in '-') ' B           # Stato prima
Passo 2: B ' (sign=-1) ' C          # Output quando entriamo in B
Passo 3: C ' (in '1') ' C           # Stato prima
Passo 4: C ' (value+=1) ' C         # Output quando rimaniamo in C
...
```

Ottimizzazioni

1. Compilare la Tabella di Transizione

Invece di usare una switch expression a runtime, precalcola tutte le transizioni:

```
delegate (State, Action) Transition;

Transition[,] table = new Transition[10, 256];

// Precompila
table[A, '+'] = (B, () => sign = -1);
table[A, '-'] = (B, () => sign = -1);
// ...
```

Beneficio: $O(1)$ lookup invece di $O(\text{pattern matching})$

2. Usare Vettori Invece di Enum

```
const int STATE_A = 0, STATE_B = 1, /* ... */ STATE_Z = 9;
int[] states = new int[256]; // Pre-allocati
```

Beneficio: Cache locality migliore

3. Parallelizzare Parsing di Batch

Se devi parsare milioni di numeri:

```
var numbers = File.ReadAllLines("large_file.txt");
var results = numbers.AsParallel()
    .Select(ParseDouble)
    .ToArray();
```

Beneficio: Sfrutta multi-core

4. Caching dei Risultati

```
Dictionary<string, double> cache = new();

double ParseDoubleWithCache(string input) {
    if (cache.TryGetValue(input, out var cached)) {
        return cached;
    }
    double result = ParseDouble(input);
    cache[input] = result;
    return result;
}
```

Beneficio: Velocizza duplicati

5. SIMD per Input Lunghi

Per parsare numerosissimi numeri corti in parallelo, usa:

- `Vector<int>` per parallelizzazione a livello bit
- `Span<T>` per evitare allocazioni heap

Conclusione

Questa macchina a stati di Mealy rappresenta un approccio elegante e efficiente per il parsing di numeri. I punti salienti:

Deterministica: per ogni (stato, input) c'è una transizione univoca

Completa: gestisce tutti i formati IEEE 754 standard

Veloce: $O(n)$ dove n è la lunghezza della stringa

Estendibile: aggiungere nuovi formati è facile (aggiungi stati/transizioni)

Applicazioni Reali

- **Parser CSV:** valida e converte numeri durante lettura

- **API validation:** verifica input utente
- **Financial software:** parsing di quantità monetarie
- **Scientific computing:** parsing di dati sperimentali
- **Compiler:** fase di tokenization numerica

Lecture Consigliate

- Hopcroft & Ullman, "Introduction to Automata Theory"
- IEEE 754 Floating-Point Standard
- Regular Expressions (superset di FSM)
- Lexical Analysis (applicazione pratica)

Documento creato: 2024

Versione: 1.0

Licenza: MIT